

Лекция 6.

Мониторинг и анализ производительности Windows Server

Цель лекции

Понимать разницу между monitoring, alerting, diagnostics, baseline и capacity planning

Читать события Windows с учётом provider, level, Event ID, времени и соседних событий

Выбирать counters CPU, memory, disk и network без универсальных ложных порогов

Связывать пользовательский симптом с измерениями, журналами, гипотезами и действием

Учебные вопросы

1. Monitoring, diagnostics и baseline; CPU, RAM, disk, network.
2. Event Logs, levels, Event ID, filters и Custom Views.
3. Task Manager, Resource Monitor и Performance Monitor.
4. Counters CPU, memory, paging, disk и network.
5. Data Collector Sets, thresholds и centralized events.
6. Симптом нагрузки и первопричина.

Учебный сценарий лекции

FS01 используется как основной сервер для baseline и моделирования нагрузки

ADMIN01 или MON01 используется как рабочее место администратора

WEC01 показывает идею централизованного сбора событий через WEF/WEC

Отчёт оформляется как цепочка: symptom -> metrics -> events -> hypothesis -> action

1. Monitoring начинается до инцидента

Администратор не должен ждать жалобы пользователя, чтобы впервые посмотреть показатели сервера

Monitoring — постоянное наблюдение за состоянием ОС, служб и ресурсов

Diagnostics — поиск причины уже замеченной проблемы

Baseline — зафиксированное нормальное поведение сервера в понятных условиях

Alerting и capacity planning дополняют monitoring

Alerting — уведомление, когда показатель или событие вышли за заданное условие

Capacity Planning — прогноз, когда CPU, RAM, disk или network перестанут хватать

Monitoring отвечает на вопрос «что происходит сейчас и во времени»

Capacity planning отвечает на вопрос «когда нужен апгрейд или изменение архитектуры»

Baseline должен иметь контекст

Пример: Server FS01, роль File Server, обычный рабочий день, 09:00-17:00

Фиксируются пользователи, backup window, обновления, роль сервера и тип нагрузки

Показатели записываются как average, peak и длительность отклонения

Одно короткое измерение не является полноценным baseline

Baseline собирают в разные периоды

Рабочее время показывает реальную
пользовательскую нагрузку

Ночное время показывает backup, обслуживание,
обновления и фоновые задачи

Пиковые периоды помогают увидеть пределы роли
сервера

Высокий disk I/O в 02:00 может быть нормой, если в
это время выполняется backup

USE-модель делает диагностику системной

USE означает Utilization, Saturation и Errors

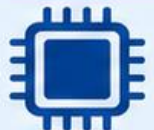
Utilization показывает, насколько ресурс занят

Saturation показывает очередь, ожидание или нехватку ресурса

Errors показывают аппаратные, драйверные или программные ошибки

USE-модель анализа ресурсов сервера

U — Utilization (занятость) • S — Saturation (перегрузка / очереди) • E — Errors (ошибки)



CPU

- U** % user, % system, % idle
- S** run queue, load average
- E** hardware errors, throttling



Для каждого ресурса задаём 3 вопроса:

- насколько занят?
- есть ли очередь?
- есть ли ошибки?



Память

- U** used / free, cache, swap usage
- S** memory pressure, swap-in / swap-out
- E** OOM, allocation failures



Диски / I/O

- U** % busy, IOPS, throughput
- S** await, avg queue length
- E** I/O errors, bad blocks



Сеть

- U** bandwidth, packets/s
- S** interface queue, drops
- E** CRC errors, retransmits, packet loss



Как читать USE-модель

- U** Высокая U: ресурс активно используется
- S** Высокая S: есть дефицит или очередь
- E** Ненулевые E: ищем сбои и неисправности



Практика



Сначала проверяют все ресурсы по схеме **U** → **S** → **E**, затем ищут узкое место.

CPU, RAM, disk и network анализируются вместе

Высокий CPU может быть самостоятельной причиной деградации

Высокая disk latency может существовать одновременно и указывать на другую проблему

Ожидание обычного I/O чаще блокирует поток, а не напрямую повышает CPU

Косвенный рост CPU возможен из-за retry, логирования, шифрования или обработки ошибок

2. Event Log хранит технический контекст инцидента

Event Log — журнал событий Windows с полями времени, источника, уровня и сообщения

Event ID без provider не является уникальным объяснением проблемы

Один и тот же Event ID у разных providers может означать разные события

Событие нужно сопоставлять с симптомом, изменением и соседними событиями

Поля события, которые нельзя игнорировать

Log Name показывает, в каком журнале находится событие

Provider или Source показывает компонент, который создал событие

Task Category и Keywords уточняют тип операции

Computer, User, TimeCreated и Correlation ID помогают связать события между собой

Уровни событий Windows

Level 1 Critical — критический сбой, требующий срочного внимания

Level 2 Error — ошибка, которая могла нарушить работу компонента

Level 3 Warning — предупреждение, часто объясняющее будущую деградацию

Level 4 Information и Level 5 Verbose используются для обычных и подробных событий



Поля события Windows Event Log

Какие данные содержит запись журнала Windows и как её читать



1 Где произошло событие?



- Журнал
- Источник

Показывают, в каком журнале записано событие и какой компонент его создал.

2 Что это за событие?



- Код события (Event ID)
- Уровень
- Категория задачи

Помогают понять тип события и его важность.

3 Когда и на каком узле?



- Дата и время
- Компьютер

Нужны для привязки события ко времени и конкретному серверу/ПК.



Событие 4625 — Неудачный вход в систему



Журнал : Security



Источник : Microsoft-Windows-Security-Auditing



Код события (Event ID) : 4625



Уровень : Audit Failure



Категория задачи : Logon



Дата и время : 12.09.2026 10:41:25



Компьютер : SRV-01.corp.local



Пользователь : SYSTEM



Security context : SYSTEM



Ключевые слова : Audit Failure



Код операции (Opcode) : Info



Описание : Неудачная попытка входа в систему



XML / Подробности : структурированные данные события

4 Кто связан с событием?



- Пользователь
- Security context

Показывает учётную запись, от имени которой произошло действие.

5 Что искать при анализе?



- Описание
- Ключевые слова
- Opcode
- XML / Подробности

Описание даёт общий смысл, XML и подробности нужны для точного разбора.



Быстрая памятка



Event ID — главный ориентир для поиска документации и типовых причин



Уровень показывает серьёзность: Information, Warning, Error, Audit Success, Audit Failure



Источник помогает понять, какая служба или подсистема сработала



Описание и XML нужны для детального расследования



Практика

1



Определить журнал и источник

2



Посмотреть Event ID и уровень

3



Проверить время, компьютер и пользователя

4



Изучить описание и подробности

Get-WinEvent выбирает события по точным условиям

Команда: Get-WinEvent — читает журналы событий Windows через PowerShell

Пример: Get-WinEvent -FilterHashtable

```
@{LogName='System'; Level=1,2; StartTime=(Get-Date).AddHours(-24)}
```

Ожидаемый результат: Critical и Error из System Log за последние 24 часа

Для системных журналов консоль лучше запускать от имени администратора

Фильтр по provider и Event ID уменьшает шум

ProviderName ограничивает выборку конкретным компонентом Windows

Пример: Get-WinEvent -FilterHashtable

```
@{LogName='System'; ProviderName='Service Control  
Manager'; Id=7031,7034; StartTime=(Get-  
Date).AddHours(-2)}
```

Ожидаемый результат: события падения или аварийного завершения служб

Такой запрос полезнее, чем просмотр всех ошибок журнала System

Дисковые события ищут по нескольким providers

Для диска проверяют providers disk, storport и Ntfs

Пример: `Get-WinEvent -FilterHashtable`

`@{LogName='System';`

`ProviderName='disk','storport','Ntfs'; Level=1,2,3;`

`StartTime=(Get-Date).AddHours(-24)}`

Ожидаемый результат: критические события, ошибки и предупреждения дисковой подсистемы

Набор providers нужно проверить на конкретном образе Windows Server

Экспорт событий делает отчёт повторяемым

Скриншот журнала хуже, чем файл с полями события

Пример: `Get-WinEvent -FilterHashtable`

`@{LogName='System'; Level=1,2,3; StartTime=(Get-Date).AddHours(-1)} | Select-Object`

`TimeCreated,Id,ProviderName,LevelDisplayName,Message | Export-Csv C:\Temp\system-events.csv -NoTypeInfoInformation -Encoding UTF8`

Ожидаемый результат: CSV-файл с событиями для отчёта

Event Viewer также позволяет сохранить исходный журнал в формате .evtx

Custom View создаётся под диагностическую задачу

Custom View — сохранённое представление Event Viewer с выбранными фильтрами

Пример Service failures: System, Service Control Manager, Event ID 7031 и 7034, Level Error

Пример Disk warnings: System, disk/Ntfs/storport, Levels Critical, Error и Warning

Сохранённый view помогает другому администратору повторить проверку

3. Task Manager полезен только для быстрой оценки

Task Manager показывает текущие процессы, users, CPU, memory, disk и network

Он помогает быстро увидеть явный процесс-лидер по нагрузке

Короткий пик легко пропустить, потому что история почти не сохраняется

Task Manager не подходит для Server Core и не заменяет PerfMon

Server Core требует консольных альтернатив

Команда: `Get-Process` — показывает процессы без графической оболочки

Пример: `Get-Process | Sort-Object -Property CPU - Descending | Select-Object -First 10`

Команда: `Get-Counter` — читает performance counters из PowerShell

Пример: `Get-Counter '\Processor(_Total)\% Processor Time'`

Resource Monitor связывает процесс с ресурсом

Resource Monitor показывает Disk Activity, TCP Connections, Listening Ports и Associated Handles
Он помогает понять, какой процесс читает файл или держит сетевое соединение

Hard Faults/sec означает обращение к странице, которой нет в RAM

Hard Fault не равен ошибке оборудования или повреждению памяти

Performance Monitor работает с объектом, счётчиком и экземпляром

Путь counter обычно выглядит как

`\Object(Instance)\Counter`

Пример: `\Processor(_Total)\% Processor Time`

Object задаёт область измерения, counter — показатель, instance — конкретный объект

На русской Windows локализованные имена counters могут отличаться, поэтому команды проверяют на образе



Путь счётчика Performance Monitor

Как читается имя счётчика в Windows Performance Monitor



1. Компьютер

- Имя локального или удалённого узла
- Может отсутствовать для локального ПК
- Пример: "SRV-01"



Показывает,
с какого сервера
берутся данные.



2. Объект

- Группа счётчиков
- Описывает подсистему Windows
- Примеры: Processor, Memory, LogicalDisk, Network Interface



Объект
определяет,
какой ресурс
анализируется.

\\Компьютер \ Объект (Экземпляр) \ Счётчик

\\SRV-01 \ Processor (_Total) \ % Processor Time



3. Экземпляр

- Конкретный экземпляр внутри объекта
- Нужен, если объектов несколько
- Примеры: _Total, C:, Ethernet0, chrome#1



Если экземпляра
нет, скобки могут
отсутствовать.



4. Счётчик

- Измеряемый показатель
- Показывает значение метрики
- Примеры: % Processor Time, Available MBytes, Disk Reads/sec



Это итоговая
метрика,
которую
показывает
PerfMon.



Как читать путь

1



1. Определить
сервер

2



2. Найти
объект

3



3. Уточнить
экземпляр

4



4. Посмотреть
сам счётчик



Важно



Обратный слеш разделяет
части пути



Экземпляр записывается
в круглых скобках



Не у всех объектов
есть экземпляры



Практика

- \\SRV-01\\Memory\\Available MBytes
- \\SRV-01\\LogicalDisk(C:)\\Avg. Disk Queue Length
- \\SRV-01\\Network Interface (Intel[R] Ethernet)\\Bytes Total/sec



Сначала читают объект и счётчик,
затем уточняют компьютер и экземпляр.

4. CPU counters показывают занятость и очередь

\Processor(_Total)\% Processor Time показывает классическую загрузку CPU

\System\Processor Queue Length показывает очередь потоков, ожидающих CPU

_Total скрывает отдельные logical processors, поэтому при спорной ситуации смотрят instances

В новых сценариях может использоваться Processor Information, но классический Processor допустим для базы

Processor Queue Length нельзя оценивать одной цифрой

Очередь зависит от числа logical processors,
виртуализации, типа нагрузки и длительности
Кратковременная очередь нормальна и не доказывает
перегрузку CPU
Проблема вероятна, если CPU долго близок к
максимуму, queue растёт и response time ухудшается
Правило «queue больше 2 всегда плохо» использовать
нельзя

Виртуальную машину нужно анализировать вместе с Hyper-V host

Гостевая Windows не видит всю конкуренцию за CPU, memory и storage на хосте

Причина задержки может быть в перегруженном host, vCPU oversubscription или checkpoint merge

Для Hyper-V полезны counters Hyper-V Hypervisor Virtual Processor и Hyper-V Dynamic Memory VM

Если проблема видна только в VM, проверка host всё равно обязательна

Memory counters нельзя читать по одному показателю

\Memory\Available MBytes показывает доступную физическую память

\Memory\% Committed Bytes In Use показывает использование commit относительно лимита

\Paging File(_Total)\% Usage показывает использование page file

Низкая Available RAM оценивается с учётом общей RAM, роли сервера, cache и длительности

Pages/sec не доказывает нехватку RAM сам по себе

\Memory\Pages/sec показывает чтение и запись страниц для hard page faults

Значение может включать page file, memory-mapped files, загрузку DLL и работу cache

Для вывода нужны Available MBytes, Committed Bytes, Page Reads/sec и Page Writes/sec

Высокий Pages/sec без контекста нельзя трактовать как однозначную нехватку памяти

Page Faults/sec и Pages/sec различаются

Page Faults/sec включает soft faults и hard faults

Soft fault может обслужиться из standby list без обращения к диску

Pages/sec связан с физическим I/O для hard faults

Page File нельзя отключать как универсальное ускорение: он нужен для commit, crash dump и стабильности

Disk latency читается в секундах и переводится в миллисекунды

`\PhysicalDisk(*)\Avg. Disk sec/Read` возвращает
задержку чтения в секундах

`\PhysicalDisk(*)\Avg. Disk sec/Write` возвращает
задержку записи в секундах

0.005 sec = 5 ms, 0.020 sec = 20 ms, 0.100 sec = 100 ms

Универсального порога latency нет: важны HDD, SSD,
NVMe, SAN, RAID, cache и роль сервера

LogicalDisk и PhysicalDisk отвечают на разные вопросы

LogicalDisk относится к томам C:, D: и удобен для % Free Space и Free Megabytes

PhysicalDisk относится к диску, представленному ОС, и удобен для latency, queue и throughput

В виртуальной среде PhysicalDisk может быть абстракцией над VHDX, SAN или storage pool

Для заполнения диска проверяют процент и абсолютный остаток одновременно

Моделировать заполнение диска нужно безопасно

Нельзя заполнять системный C: почти до нуля

Лучше использовать отдельный тестовый том или ограниченный VHDX

После теста заранее известный test-file удаляется и проверяется Get-Volume

Сценарий нельзя проводить на диске Exchange logs, базы данных или production storage

Network throughput сравнивают со скоростью интерфейса

\Network Interface(*)\Bytes Total/sec показывает общий сетевой трафик

\Network Interface(*)\Current Bandwidth показывает скорость интерфейса в bits per second

Utilization примерно равен $\text{Bytes Total/sec} * 8 / \text{Current Bandwidth} * 100\%$

80 MB/s много для 1 Gbit/s, но обычно мало для 10 Gbit/s

Ошибки сети важны даже при нормальной скорости

Packets Received Errors и Packets Outbound Errors
показывают ошибки приема и передачи

Packets Received Discarded и Packets Outbound
Discarded показывают отброшенные пакеты

Output Queue Length помогает увидеть очередь на
отправку

Команда: Get-NetAdapterStatistics — быстро
показывает статистику адаптера

Network saturation может находиться вне сервера

Проблема может быть на switch port, uplink, WAN, VPN, firewall или load balancer

В Hyper-V отдельным местом проверки становится virtual switch

Локальный counter сервера не показывает всю сетевую цепочку

Если ошибок на сервере нет, проверяют соседние сетевые устройства и маршрут трафика

Минимальный набор counters для quick check

CPU: \Processor(_Total)\% Processor Time и
\System\Processor Queue Length

Memory: \Memory\Available MBytes, \Memory\
Committed Bytes In Use и \Memory\Pages/sec

Disk: \PhysicalDisk(_Total)\Avg. Disk sec/Read и Avg. Disk
sec/Write

Network: \Network Interface(*)\Bytes Total/sec и
error/discard counters

Get-Counter может собрать несколько показателей

Команда: Get-Counter — собирает counters с заданным интервалом и числом samples

Пример: \$counters = @('\Processor(_Total)\% Processor Time', '\Memory\Available MBytes', '\Memory\Pages/sec', '\PhysicalDisk(_Total)\Avg. Disk sec/Read')

Пример: Get-Counter -Counter \$counters -SampleInterval 5 -MaxSamples 12

Ожидаемый результат: минута наблюдения; это quick check, но ещё не полноценный baseline

5. Data Collector Set собирает проблему во времени

Data Collector Set (DCS) — набор counters и правил записи в Performance Monitor

DCS нужен, если проблема появляется ночью, при backup или без администратора рядом

В DCS задаются counters, sample interval, duration, path, format и правила хранения

Для учебной лаборатории удобно писать BLG-файл на отдельный том

Sample Interval меняет качество данных

- 1 second ловит короткие пики, но создаёт большой log
- 5-15 seconds — хороший учебный компромисс для лаборатории
- 60 seconds подходит для длительного trend
- 5 minutes может пропустить короткий инцидент

DCS должен иметь безопасный path и format

BLG — компактный формат, который удобно открывать в PerfMon

Пример пути: D:\PerfLogs\FS01-Baseline

PerfLogs не размещают на почти заполненном системном диске

Настраивают maximum size, subdirectory format, расписание и удаление старых logs

Threshold должен учитывать baseline и длительность

Threshold — порог, при котором система записывает событие или запускает действие

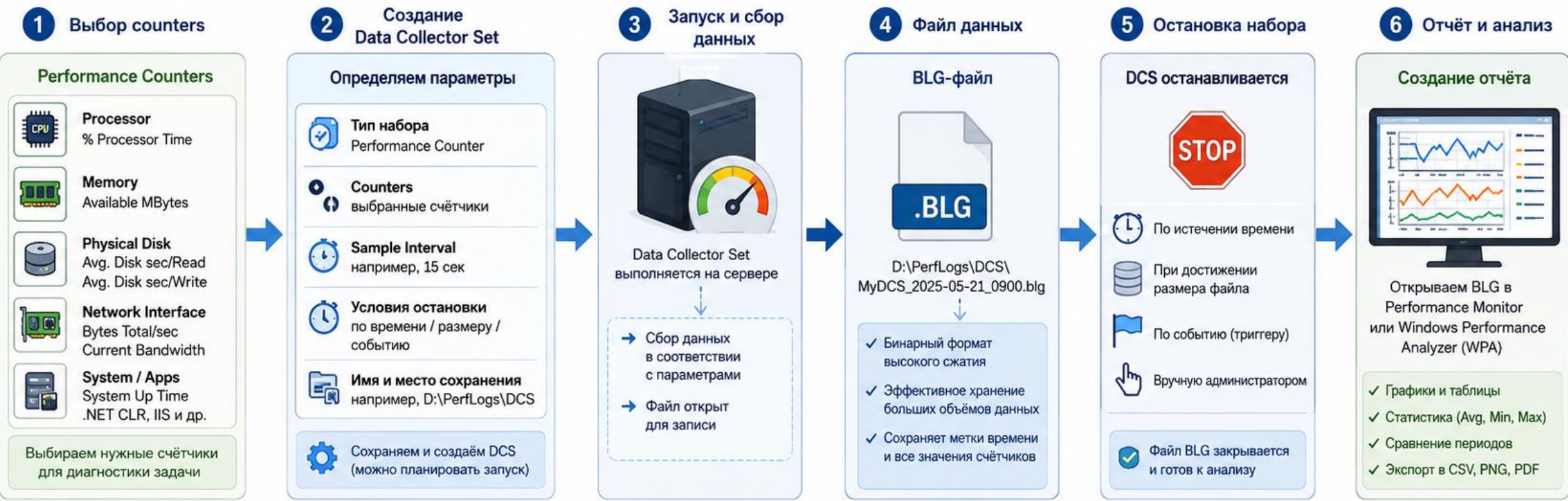
Порог CPU > 80% один раз создаёт шум и не доказывает инцидент

Практичнее условие: CPU > 80% в течение 5 минут или latency выше baseline несколько samples подряд

PerfMon threshold полезен для обучения, но не заменяет SCOM, Zabbix, PRTG или Azure Monitor

Схема: Data Collector Set от counters к BLG-отчёту

Performance Monitor → Data Collector Set → сбор данных → BLG-файл → отчёт и анализ



Centralized events в Windows — это WEF и WEC

WEF (Windows Event Forwarding) — пересылка событий с источников на collector

WEC (Windows Event Collector) — сервер, принимающий forwarded events

WinRM обеспечивает удалённое взаимодействие для подписок

Forwarded Events Log хранит события, пришедшие от source computers

Source-initiated и collector-initiated subscriptions

Collector-initiated: collector сам подключается к заданным серверам

Source-initiated: серверы получают адрес collector через GPO и отправляют события

Для домена source-initiated обычно масштабируется удобнее

Собирать все события нельзя: фильтр должен соответствовать роли и задаче

WEF требует WinRM, collector и фильтр событий

На collector выполняют `wesutil qc` для первичной настройки Event Collector

На источниках WinRM можно подготовить через GPO или командой `winrm quickconfig`

Перед лабораторией проверяют firewall, права чтения журналов и доступность WinRM

WEF не является полноценным SIEM: нет полноценной корреляции, нормализации и расследования



WEF/WEC и журнал Forwarded Events

Как Windows Event Forwarding доставляет события на сервер-сборщик

A. Источники событий



- Локальные журналы: System, Security, Application
- События выбираются по фильтру
- Источник = source computer

B. WEF / Подписка



- Подписка определяет, какие события отправлять
- Фильтр по журналу, уровню, Event ID, provider
- Доставка по доменной инфраструктуре

WinRM

Режимы доставки

Source-initiated



источники сами отправляют события по настройке через GPO

Collector-initiated



сервер-сборщик сам подключается к известным узлам

C. WEC — Windows Event Collector



WEC01

- Принимает forwarded events
- Хранит их централизованно
- Упрощает анализ на нескольких серверах

D. Журнал Forwarded Events



Forwarded Events

- Все принятые события видны в одном журнале
- Открывается в Event Viewer
- Удобен для фильтрации и расследования

E. Практика

1

На collector:
wecutil qc



2

На источниках:
включить WinRM



3

Создать подписку
и фильтр



4

Проверить журнал
Forwarded Events



F. Важно



- WEF/WEC — это централизованный сбор событий
- Это не полноценная SIEM
- Собирать нужно только нужные события, без лишнего шума

6. Симптом нагрузки ещё не является причиной

Симптом — то, что видит пользователь или служба:
медленный вход, timeout, зависание

Причина — доказанный ресурс, событие, изменение
или ошибка конфигурации

Показатель без времени и контекста не доказывает
root cause

Диагностика строится через гипотезы и проверяемые
признаки

Таблица гипотез помогает не угадывать

Симптом: медленный вход пользователя

Гипотеза DNS: проверка Resolve-DnsName и DC Locator

Гипотеза GPO: проверка gpresult и GroupPolicy events

Гипотеза disk или profile: проверка latency, User Profile events и размера профиля

Последнее изменение часто указывает направление поиска

Нужно выяснить, что изменилось, когда и кем

Проверяются updates, GPO, backup schedule,
приложение, reboot и рост нагрузки

Связка incident time и change time часто быстрее
приводит к причине

Если время изменения совпадает с началом симптома,
гипотеза получает приоритет

Моделирование CPU load должно быть безопасным

Тест выполняется только в изолированной учебной VM или на тестовом сервере

Нагрузка должна иметь ограниченную продолжительность и известную команду остановки
Baseline фиксируется до теста, а recovery проверяется после остановки нагрузки

Бесконтрольный бесконечный цикл не подходит для лабораторной работы

Отказ службы моделируют на безопасной службе

Для демонстрации можно использовать тестовую или некритичную службу, например Spooler на отдельном сервере

Пример: Stop-Service Spooler

Проверка: Get-Service Spooler

Нельзя без отдельного сценария останавливать AD DS, DNS на единственном DC, Exchange Store или Hyper-V Management

Recovery и root cause analysis решают разные задачи

Recovery быстро возвращает сервис: запустить службу, освободить место, переключить backend

Root Cause Analysis объясняет, почему проблема возникла и как не повторить её

Перезагрузка может снять симптом, но скрыть первопричину

После действия всегда выполняется повторная проверка counters и events

Мониторинг связывается с ролями предыдущих лекций

Exchange: queues, mounted databases, transaction log disk, services и certificates

WSUS: synchronization, SUSDB/IIS, content disk и client contact

Hyper-V: host RAM, vCPU, VHDX storage, checkpoints и vSwitch

File Server: disk latency, SMB sessions, free space и NTFS events

Диаграмма: от пользовательского симптома к root cause

Цель: системно пройти путь от жалобы пользователя до истинной причины и устранения проблемы.



Коротко о потоке:



Отчёт должен содержать временную шкалу

10:00 — начат baseline collection

10:05 — запущена тестовая нагрузка

10:07 — CPU достиг 92% и вырос response time

10:11 — нагрузка остановлена, показатели вернулись к baseline

Минимальная лабораторная топология

ADMIN01 или MON01: администрирование и централизованный collector

FS01: основной объект мониторинга и baseline

DC01: role-specific events и пример доменной роли

CLIENT01 и test disk: пользовательский симптом и безопасное моделирование заполнения

Измеримая приёмка мониторинга

Baseline содержит роль сервера, период, условия, counters и путь к BLG

Custom View создан под конкретную задачу и повторяет нужный фильтр

DCS имеет interval, duration, log format и безопасный log path

Отчёт содержит symptom, hypothesis, metrics, events, action и повторную проверку

Итоги лекции

Monitoring без baseline превращается в набор случайных чисел

Event ID читается только вместе с provider, временем, уровнем и соседними событиями

CPU, memory, disk и network оцениваются по utilization, saturation и errors

DCS и WEF помогают видеть проблему во времени и на нескольких серверах

Root cause подтверждается измерениями, журналами и повторной проверкой после действия

Контрольные вопросы

Чем monitoring отличается от diagnostics?

Что должно входить в baseline сервера?

Почему один Event ID нельзя оценивать без provider?

Что означает Level=2 в Get-WinEvent?

Чем Custom View отличается от обычного ручного просмотра журнала?

Контрольные вопросы: counters и диагностика

Почему высокий Pages/sec не всегда доказывает нехватку RAM?

В каких единицах измеряется Avg. Disk sec/Read?

Чем LogicalDisk отличается от PhysicalDisk?

Как определить загрузку сетевого интерфейса в процентах?

Чем WEF отличается от полноценной SIEM-системы?